

Spring Framework

Spring Core | Spring MVC | Spring Boot

Software Developer Interview — Complete Q&A Guide

Part 1 — Spring Core: Framework Fundamentals

Q1. What is Spring Framework? Why was it introduced?

Answer:

Spring is an open-source, lightweight framework for building Java applications, introduced in 2003 by a team led by Rod Johnson (originally called Interface 21).

Why was it introduced?

Before Spring, Java developers used Enterprise JavaBeans (EJBs), which had severe limitations:

- Complexity — Configuration and deployment were extremely complex.
- Heavyweight — Large runtime overhead and difficult setup.
- Difficult Testing — Tight coupling made unit testing nearly impossible.
- Rigid Architecture — Changes in one class required changes across many others.

Spring was introduced to solve these problems with a lightweight, loosely coupled, easily testable alternative.

Q2. What is Spring Core? What are its key features?

Answer:

Spring Core is the foundational module of the Spring Framework. All other Spring modules (Spring MVC, Spring Data JPA, Spring Security) are built on top of it.

Key Features:

- Inversion of Control (IoC) — The container controls object creation, wiring, and lifecycle.
- Dependency Injection (DI) — Dependencies are injected by the container, not created by the class itself.
- Modular & Flexible — Developers configure only the components they need.
- Loose Coupling — Changes in one component don't drastically affect others.
- Bean Lifecycle Management — Manages the full lifecycle of application beans (POJO objects).

Q3. What is IoC (Inversion of Control)? How does it differ from traditional Java?

Answer:

IoC stands for Inversion of Control. It is a Spring principle where the control of object creation and lifecycle management is transferred from the developer to the Spring framework.

Traditional Java approach:

- Developer creates objects with 'new', manages their lifecycle, and handles dependencies manually.

Spring IoC approach:

- The IoC Container reads configuration (XML or annotations), creates objects, assigns names, and manages the entire lifecycle.
- Developer focuses on business logic — not object management.

IoC Container is launched like: `ApplicationContext ctx = new ClassPathXmlApplicationContext("config.xml");`

The main purpose of IoC: programmer should focus on functionality and business logic only.

Q4. What is the Spring IoC Container? What are its types?

Answer:

The Spring IoC Container is responsible for bean management — controlling the entire lifecycle of POJO class objects in Spring-based applications.

- Reads XML or annotation configuration and creates/manages beans.
- Acts as an interface between the low-level JRE layer and the component layer (DAO/Service/Controller).

Types of Spring Containers:

- BeanFactory — The basic IoC container. (XmlBeanFactory is deprecated from Spring 3.1+, removed in Spring 5).
- ApplicationContext — Advanced IoC container built on top of BeanFactory. Recommended for all modern applications.
- Web Container — Built on top of IoC; used in Spring MVC for web applications.

ApplicationContext is used as: `ConfigurableApplicationContext ctx = new ClassPathXmlApplicationContext("config.xml");`

Q5. What is a Bean in Spring? How do you create one?

Answer:

A Bean is a lightweight Java object (POJO) whose lifecycle is managed by the Spring IoC container.

Creating a bean using XML config:

```
<bean id="student" class="pojo_classes.Student"></bean>
```

Creating a bean using @Component annotation:

```
@Component
public class Car { }
```

- Bean name auto-generated as 'car' (camelCase of class name).
- Can also assign custom name: @Component("myCar")

Creating a bean using @Bean on a factory method:

```
@Bean("duster")
public Duster getDuster() { return new Duster(); }
```

- Used when the class is not editable/accessible (e.g., third-party classes).

Q6. What is Dependency Injection (DI)? What are its types?

Answer:

Dependency Injection is the process where the Spring container provides (injects) required dependency objects into a dependent class — instead of the class creating them itself.
Example: Person class has a Mobile object. Mobile is injected into Person by the Spring container.

3 Types of Dependency Injection:

1. Setter Injection:

- Dependencies injected using setter methods.
- Configured with <property name="mobile" ref="mobile_id" or value="primitive/string"/> in XML, or @value, @Autowired on setter.

2. Constructor Injection:

- Dependencies injected through the constructor.
- Configured with <constructor-arg ref="mobile_id"/> in XML.
- Recommended approach for mandatory dependencies.

3. Field Injection:

- Dependencies injected directly into fields using @Autowired for objects and @value for primitive and string.
- Simplest but least recommended (hard to test).

Q7. What is @Autowired? What happens if multiple beans qualify?

Answer:

@Autowired is a Spring annotation that tells the Spring container to automatically inject the required dependency bean into the annotated field, constructor, or method.

- Can be applied to fields, setter methods, or constructors.

Problem with multiple qualifying beans:

- If two classes implement the same interface and both are annotated with `@Component`, Spring gets confused and throws:
- `UnsatisfiedDependencyException`: expected single matching bean but found 2.

Solutions:

`@Qualifier("beanName")`:

- Used with the injection point to specify exactly which bean to inject.

```
@Autowired
@Qualifier("plasticRuler")
private Ruler ruler;
```

`@Primary`:

- Class-level annotation on one of the qualifying classes.
- Spring automatically chooses the `@Primary` bean when multiple qualify.

Q8. What is `@Component`, `@Configuration`, and `@ComponentScan`?

Answer:

`@Component`:

- Class-level annotation that marks a POJO for Spring to manage as a bean.
- Replaces the `<bean>` XML tag.
- Specialized forms: `@Service` (business layer), `@Repository` (DAO layer), `@Controller` (web layer).

`@Configuration`:

- Class-level annotation indicating the class contains one or more `@Bean` factory methods.
- Used to completely replace XML configuration files.

`@ComponentScan("base.package")`:

- Tells the Spring container where to look for `@Component`-annotated classes.
- Used alongside `@Configuration`.

```
@Configuration
@ComponentScan("pojo_classes")
public class Config { }
```

Note: With `@Configuration` + `@ComponentScan`, no XML config file is needed at all.

Q9. What are Bean Scopes in Spring? Explain Singleton vs Prototype.

Answer:

Bean Scope defines how many instances of a bean Spring creates and how long they live. Configured using `@Scope` annotation on class level or method level.

Singleton (Default):

- Only ONE instance is created for the entire Spring container.
- All `getBean()` calls return the same object (same reference/address).
- Bean is created eagerly at context startup.
- Use for stateless beans: Services, Repositories, DAOs, Controllers, Utilities.

```
@Component @Scope("singleton") // or just @Component
```

Prototype:

- A NEW instance is created every time `getBean()` is called.
- All `getBean()` calls return different objects (different references).
- Bean is created lazily — only when requested.
- Use for stateful beans that hold request-specific or session-specific data.

```
@Component @Scope("prototype")
```

Key rule: Singleton = memory-efficient, shared logic. Prototype = fresh instance, per-request data.

Q10. What is the Spring Bean Lifecycle? What are init-method and destroy-method?

Answer:

Every Spring-managed bean goes through these phases:

1. Instantiation — Object of the bean class is created.
2. Dependency Injection — Dependencies are injected by the container.
3. Initialization (Post-Construct) — Custom init logic runs after DI is complete.
4. Ready for Use — Bean is available to the application.
5. Destruction (Pre-Destroy) — Custom cleanup logic runs before the container shuts down.

init-method (Post-Construct phase):

- Called after Spring finishes dependency injection.
- Used for: initializing resources, validating data, starting connections.

destroy-method (Pre-Destroy phase):

- Called before the bean is destroyed (when the container closes).
- Used for: releasing resources, closing connections, cleanup.

```
<bean id="myBean" class="MyClass" init-method="postConstruct" destroy-  
method="preDestruct"/>
```

Part 2 — Spring MVC: Web Application Development

Q11. What is Spring MVC? Why is it used?

Answer:

Spring MVC is a sub-framework of Spring Framework used to develop web applications. It is built on top of the Servlet API and follows the Model-View-Controller (MVC) design pattern.

- Internally uses Servlet API.

- Implements Spring Core features like IoC and DI.

Why use Spring MVC?

- Separates responsibilities: Model (data), View (UI), Controller (logic).
- Powerful, flexible configuration.
- Leverages full Spring Core ecosystem.
- Rapid Application Development.
- Easily testable and feature-rich.

Q12. What is the MVC Design Pattern? Explain with a real-world analogy.

Answer:

MVC (Model-View-Controller) is a design pattern that organizes code into three distinct layers:

- Model — Business logic and data (Java objects, Services, DAO layer).
- View — The UI layer (JSP, Thymeleaf, HTML).
- Controller — Handles user requests, calls the Model, returns a View.

Restaurant Analogy:

- Controller = Waiter — Takes the customer's order, passes it to the kitchen, brings the food back.
- Model = Chef — Prepares the food (business logic) behind the scenes.
- View = Dining Area — Presents the final result (food) to the customer.

Without MVC (problem):

A single Servlet/JSP contains UI + business logic + DB code all mixed — unmanageable as the system grows.

With MVC (solution):

Each layer has ONE responsibility — clean, maintainable, and testable code.

Q13. Explain the complete request-response flow in Spring MVC.

Answer:

Step 1: Client sends a request (e.g., GET /hello).

Step 2: DispatcherServlet (Front Controller) receives ALL requests — it is the single entry point.

Step 3: HandlerMapping identifies which Controller method handles this URL.

Step 4: The mapped Controller method is called; it processes the request and calls Service/DAO (Model) if needed.

Step 5: Controller returns a logical view name + model data to DispatcherServlet.

Step 6: ViewResolver maps the logical view name to the actual file (e.g., 'hello' → /WEB-INF/views/hello.jsp).

Step 7: The View (JSP/Thymeleaf) renders the UI using the model data.

Step 8: DispatcherServlet sends the final response back to the client.

Q14. What is DispatcherServlet? What is its role?

Answer:

DispatcherServlet is the Front Controller in Spring MVC. It is a single servlet that receives ALL incoming HTTP requests and coordinates the entire request-handling process.

Responsibilities:

- Intercepts every request at the entry point (mapped to '/' in web.xml).
- Delegates to HandlerMapping to find the right Controller.
- Delegates to ViewResolver to find the right View file.
- Sends the final response back to the client.

Configuration in web.xml:

```
<servlet>
  <servlet-name>spring</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>spring</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

Q15. What are @Controller, @RequestMapping, Model, and ModelAndView?

Answer:

@Controller:

- Marks a class as a Spring MVC controller (handles user requests).
- Also enables auto-scanning of the class.

@RequestMapping:

- Maps an HTTP request (URL) to a specific controller method.
- Can be used at class level (base URL) and method level (specific endpoint).

```
@RequestMapping("/add")
public String add() { return "output"; }
```

Model interface:

- Passes data from Controller to View.
- addAttribute(key, value) — stores data to be displayed in JSP using Expression Language \${key}.

ModelAndView class:

- Passes BOTH the view name and the data together.
- setViewName("filename") — specifies the view to render.
- addObject(key, value) — adds data to the model.

Use Model when view name is returned as String. Use ModelAndView when both data and view are needed together.

Q16. How do you send data from View to Controller? Explain @RequestParam, @PathVariable, and @ModelAttribute.

Answer:

@RequestParam:

- Extracts query parameters from the URL or form data.
- URL format: /add?first=10&second=20

```
public ModelAndView add(@RequestParam("first") int num1, @RequestParam("second") int num2)
```

- Can be made optional: @RequestParam(required = false)

@PathVariable:

- Extracts dynamic values embedded directly in the URL path.
- URL format: /add/10/20

```
@RequestMapping("/add/{n1}/{n2}")
public ModelAndView add(@PathVariable("n1") int n1, @PathVariable("n2") int n2)
```

- Always mandatory — defines the resource identity.

@ModelAttribute (variable level):

- Fetches form data and automatically maps it to a POJO object using setters.
- Input field names in the form must match variable names in the class.

```
public ModelAndView add(@ModelAttribute Addition info)
```

@ModelAttribute (method level):

- Pre-populates model attributes before any controller method runs.

Part 3 — Spring Boot: Auto-Configuration & REST APIs

Q17. What is Spring Boot? How is it different from Spring MVC?

Answer:

Spring Boot is a powerful, open-source framework built on top of the Spring Framework that simplifies and accelerates the creation of production-ready, stand-alone Spring applications with minimal manual configuration.

- Provides auto-configuration — no boilerplate XML configuration needed.
- Every Spring Boot project has an auto-configured embedded server (Apache Tomcat by default).
- Provides Spring Data JPA (built on top of JPA/Hibernate) out of the box.
- Majorly used for RESTful APIs, microservices, and backend services.

Spring Boot vs Spring MVC:

- Spring MVC: Requires manual configuration (web.xml, servlet config, view resolver setup).
- Spring Boot: Auto-configures everything; uses @SpringBootApplication as the entry point.
- Spring MVC returns HTML views (JSP/Thymeleaf).
- Spring Boot primarily returns JSON data (RESTful APIs).

Q18. What is @SpringBootApplication? What annotations does it combine?

Answer:

@SpringBootApplication is the main entry-point annotation for any Spring Boot application. It is placed on the main class (Driver class).

It is a combination of THREE annotations:

- @SpringBootConfiguration — Marks the class as a Spring configuration class.
- @ComponentScan — Automatically scans all packages for @Component, @Service, @Repository, @Controller classes.
- @EnableAutoConfiguration — Enables Spring Boot's automatic configuration based on dependencies on the classpath.

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Q19. What is a Web Service? What is the difference between SOAP and REST?

Answer:

A Web Service is a server-side application that provides a service (only data, not UI) to multiple clients over the web.

SOAP (Simple Object Access Protocol):

- A protocol — strict set of rules to follow.
- Data format: XML only — heavyweight and verbose.
- Considered rigid; harder to work with.

REST (Representational State Transfer):

- An architectural style — not a protocol; flexible.
- Data format: JSON (lightweight) — also supports XML.
- Stateless: each request contains all necessary information.
- Cacheable and fast.
- Default choice for modern APIs — widely used in Spring Boot.

REST is preferred over SOAP because JSON is lighter, faster, and more developer-friendly than XML.

Q20. What is a Microservice? How does it differ from a Web Service?

Answer:

A Microservice is a web service itself, but at a small scale.

Web Service (Monolithic):

- A single server-side application handling all functionality.
- If one part fails or gets overloaded, the entire application is affected.

Microservice Architecture:

- The server-side application is divided into smaller, independent, loosely coupled modules (services).
- Each service runs in its own process, can be developed and deployed independently.
- Services communicate via APIs (usually REST).

Real-Time Example (E-commerce):

- User Authentication Service
- Product Catalog Service
- Shopping Cart Service
- Order Processing Service
- If Product Catalog gets heavy traffic, only that service is scaled — others are unaffected.

Q21. What is JSON? Why is it preferred for data exchange in REST APIs?**Answer:**

JSON (JavaScript Object Notation) is a lightweight, language-independent data format used to store and transmit data between client and server.

JSON Structure:

- Objects: key-value pairs enclosed in {} — e.g., {"name": "Alex", "age": 30}
- Arrays: ordered lists enclosed in [] — e.g., ["apple", "banana"]

JSON Supported Data Types:

- String, Number, Boolean (true/false), Object, Array, null

Why JSON is preferred:

- Simple and human-readable structure.
- Lightweight — less verbose than XML, smaller file sizes.
- Language-independent — works with Java, Python, JavaScript, etc.
- Fast to parse — built-in support in most languages.
- Ideal for high-performance web applications.

Q22. What is @RestController? How is it different from @Controller?**Answer:****@Controller:**

- Used in Spring MVC to return HTML views (JSP, Thymeleaf pages).
- Returns a view name (String) that ViewResolver maps to a file.

- Suitable for traditional web MVC applications.

@RestController:

- Combination of @Controller + @ResponseBody.
- Returns data directly (JSON/XML/text) in the HTTP response body — not a view.
- Spring Boot uses this to build RESTful web services.

@ResponseBody:

- Tells Spring to serialize the returned Java object into JSON/XML using the Jackson library.
- This process is called Marshalling (Java Object → JSON format).

```
@RestController
@RequestMapping("/api")
public class ProductController { ... }
```

Q23. What are the HTTP method mapping annotations in Spring Boot?

Answer:

Spring Boot provides dedicated annotations for each HTTP method — they are shortcuts for @RequestMapping(method = RequestMethod.X):

- @GetMapping — Fetches/retrieves data. GET /products
- @PostMapping — Sends/creates data. POST /products
- @PutMapping — Fully updates existing data. PUT /products/1
- @PatchMapping — Partially updates some fields. PATCH /products/1
- @DeleteMapping — Deletes existing data. DELETE /products/1

Note: Only GET requests can be tested via a browser. POST, PUT, PATCH, DELETE must be tested using tools like Postman.

@RequestMapping can still be used at class level to define a base URL for all methods:

```
@RequestMapping("/products")
```

Q24. What is the difference between @RequestParam and @PathVariable in Spring Boot?

Answer:

@PathVariable:

- Extracts values embedded inside the URI path itself.
- URL format: /employees/10 → the '10' is a path segment.
- Usually mandatory — defines the identity of the resource.
- Used in RESTful APIs to identify a specific resource.

```
@GetMapping("/user/{id}")
public String getUser(@PathVariable("id") int userId) { ... }
```

@RequestParam:

- Extracts values from the query string (after '?' in the URL).
- URL format: /employees?dept=HR&sort=salary
- Can be made optional: @RequestParam(required = false).
- Can have default values: @RequestParam(defaultValue = "10").
- Used for filtering, sorting, searching, or pagination parameters.

```
@GetMapping("/search")
public String search(@RequestParam String name) { ... }
```

Rule: Use @PathVariable to identify a resource. Use @RequestParam for optional filters/options.

Q25. What are @RequestBody and @ResponseBody? What is Marshalling and Unmarshalling?

Answer:

@RequestBody:

- Annotation applied to a method parameter that binds the HTTP request body to a Java object.
- Used for POST/PUT requests that send JSON data in the request body.
- Spring uses the Jackson library to convert JSON → Java object. This is called Unmarshalling (Deserialization).

```
@PostMapping("/student")
public String save(@RequestBody Student student) { ... }
```

@ResponseBody:

- Tells Spring that the method's return value should be written directly to the HTTP response body.
- Spring converts the Java object → JSON. This is called Marshalling (Serialization).
- Automatically included when @RestController is used.

Summary:

- Marshalling = Java Object → JSON (for HTTP response)
- Unmarshalling = JSON → Java Object (from HTTP request body)

Q26. What is the Layered Architecture in Spring Boot?

Answer:

Layered Architecture is a software design pattern that divides a Spring Boot application into separate layers — each with a distinct responsibility.

Layers (top to bottom):

- Controller Layer — Handles incoming HTTP requests. Returns JSON responses. Like a waiter.
- Service Layer (@Service) — Contains business logic. Like a chef.
- Repository Layer (@Repository) — Data access (CRUD operations on DB). Like kitchen storage.
- Model/Entity Layer (@Entity) — Represents database tables as Java objects.
- DTO Layer — Data Transfer Objects used between client ↔ controller (hides internal entity details).

Why use Layered Architecture?

- Separation of Concerns — each layer has one job.
- Easier to debug — know exactly where logic lives.
- More modular and extensible.
- Enterprise-ready standard for large teams.

Q27. What is JpaRepository? What built-in methods does it provide?

Answer:

JpaRepository<T, ID> is the central Spring Data JPA repository interface. When your repository interface extends it, Spring automatically generates a full JPA-based data-access implementation — no boilerplate EntityManager code needed.

- T = Entity type (e.g., Product)
- ID = Primary key data type (non-primitive, e.g., Integer)

Built-in Methods provided:

- save(entity) — Insert or update (if ID exists).
- findById(ID) — Fetch one entity by primary key (returns Optional<T>).
- findAll() — Fetch all records.
- deleteById(ID) — Delete by primary key.
- count() — Returns total number of records.
- existsById(ID) — Returns true/false.

It also supports custom query methods by naming convention and @Query annotation.

Q28. What are Custom Repository Methods in Spring Data JPA? Explain naming convention rules.

Answer:

Spring Data JPA allows you to create custom query methods by following a naming convention — Spring automatically generates the SQL query from the method name.

Rule 1 — Start with the right prefix:

- findBy / readBy / getBy → SELECT records
- countBy → COUNT records
- existsBy → Check if record exists (boolean)
- deleteBy → DELETE records

Rule 2 — Use exact entity field names:

```
List<Product> findByName(String name);  
// → SELECT * FROM product WHERE name = ?  
List<Product> findByPriceGreaterThan(double price);  
// → SELECT * FROM product WHERE price > ?
```

Common keywords:

- LessThan, GreaterThan, Between, Like, In, IsNull, IsNotNull
- StartingWith, EndingWith, Containing, True, False

Custom JPQL with @Query:

```
@Query("SELECT p FROM Product p WHERE p.name = :name AND p.price < :price")
List<Product> findProductsUnderPrice(@Param("name") String name, @Param("price")
double price);
```

Q29. What is ResponseEntity? Why use it over returning just a response body?(Extra)**Answer:**

ResponseEntity<T> represents the COMPLETE HTTP response — it gives full control over all three components:

- HTTP Status Code (e.g., 200 OK, 404 NOT_FOUND, 400 BAD_REQUEST)
- HTTP Headers
- Response Body

Why use it?

- Returning just an object (e.g., ApiResponseDTO) only controls the response body — the actual HTTP status code shown to the client is always 200.
- ResponseEntity lets you set the REAL HTTP status code along with the body.

Example:

```
return new ResponseEntity<>(response, HttpStatus.OK);
return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
```

Combined with a custom ApiResponseDTO<T>:

```
public class ApiResponseDTO<T> {
    private int statusCode;
    private String message;
    private T data;
}
```

This gives full control over both the response structure AND the actual HTTP status.

Q30. What is Global Exception Handling in Spring Boot? Explain @RestControllerAdvice and @ExceptionHandler.**Answer:**

Spring Boot allows you to centralize all exception handling logic in one class using @RestControllerAdvice.

@RestControllerAdvice:

- Makes a class a global exception handler — catches exceptions from ALL controllers.

- Equivalent to `@ControllerAdvice` + `@ResponseBody`.

@ExceptionHandler(SomeException.class):

- Annotates a method inside the advice class.
- Automatically triggered when the specified exception type is thrown anywhere in the application.

Benefits:

- Consistency — all errors return the same structured JSON format.
- Decoupling — error handling logic is separated from business logic in controllers.
- Predictability — clients always get a clean, predictable error response instead of a server error page.

Example Error Response:

```
{ "timestamp": "2025-11-07T15:46:10",  
  "statusCode": 400,  
  "error": "Constraint Violation!",  
  "message": "not-null property references null" }
```

Q31. What is Swagger UI in Spring Boot? Why is it used?(Extra)

Answer:

Swagger UI is an interactive, browser-based documentation and testing tool for RESTful APIs in Spring Boot.

- Presents all API endpoints in a clean, user-friendly interface.
- Allows developers and non-developers to understand, interact with, and test APIs with minimal effort.
- Integrated via SpringDoc OpenAPI dependency in pom.xml.
- Accessible at: <http://localhost:8080/swagger-ui/index.html>

Swagger Annotations:

- `@Tag(name, description)` — Describes the complete controller/API group.
- `@Operation(summary, description)` — Describes a specific endpoint/method.
- `@ApiResponse(responseCode, description)` — Documents possible response codes.
- `@ApiResponses` — Groups multiple `@ApiResponse` annotations.
- `@Content` + `@Schema` — Specifies the response structure for a given response code.

Part 4 — Scenario-Based & Comparison Questions

Q32. What is the difference between @Service, @Repository, @Controller, and @Component?

Answer:

All four are specializations of `@Component` — they all register the class as a Spring-managed bean. The difference is in their semantic role and additional behaviours:

- **@Component** — Generic bean marker. Use for classes that don't fit other layers.
- **@Service** — Marks a class as the Service/Business Logic layer. No extra behaviour — purely semantic.
- **@Repository** — Marks a class as the Data Access/DAO layer. Additionally, Spring wraps checked persistence exceptions into unchecked Spring `DataAccessExceptions`.
- **@Controller** — Marks a class as the Web/Presentation layer controller. Enables view resolution.
- **@RestController** — **@Controller** + **@ResponseBody**. Returns JSON/XML directly.

Q33. What is the difference between PUT and PATCH HTTP methods?

Answer:

PUT:

- Fully replaces an existing resource.
- The request body must contain the complete updated object.
- If a field is not included in the request, it is set to null/default.
- Example: Update ALL fields of a product.

PATCH:

- Partially updates an existing resource.
- The request body contains ONLY the fields that need to be updated.
- Other fields remain unchanged in the database.
- Example: Update only the price of a product, keeping name and id unchanged.

Rule: Use PUT for full replacement. Use PATCH for partial modification.

Q34. What is the difference between @RequestHeader, @CookieValue, and @RequestBody?

Answer:

@RequestHeader:

- Extracts HTTP header information from the request (metadata about the request).
- Headers carry auth tokens, content type, caching info, connection settings, etc.

```
public String test(@RequestHeader("token") String authToken)
```

@CookieValue:

- Extracts a specific cookie value from the HTTP request.
- Cookies are small pieces of data stored on the client's side.

```
public String info(@CookieValue("name") String productName)
```

@RequestBody:

- Binds the entire HTTP request body (JSON/XML) to a Java object.
- Used for POST and PUT requests that send complex data.
- Jackson library performs unmarshalling (JSON → Java Object).

```
public String save(@RequestBody Product product)
```

Q35. What is Media Type in Spring Boot? Explain consumes and produces.(Extra)

Answer:

Media Type defines the format of the request body and/or response body in an HTTP communication.

- Common media types: JSON, XML, plain text, form data, CSV, file.
- The default MediaType for @RestController is JSON.

consumes attribute:

- Declares which media type the controller method ACCEPTS in the request body.
`@PostMapping(value="/add", consumes=MediaType.APPLICATION_JSON_VALUE)`
- If the client sends a different format, a 415 Unsupported Media Type error is returned.

produces attribute:

- Declares which media type the controller method RETURNS in the response body.
`@PostMapping(value="/add", produces=MediaType.APPLICATION_XML_VALUE)`
- To produce XML responses, add Jackson Dataformat XML dependency to pom.xml.

When neither is specified, JSON is used by default for both request and response.

Q36. What is the difference between Spring MVC @Controller and Spring Boot @RestController?

Answer:

Spring MVC @Controller:

- Returns an HTML view name (String) → ViewResolver maps to JSP/Thymeleaf file.
- Used for traditional server-side rendered web applications.
- Requires view template files (JSP, HTML) in the WEB-INF folder.
- Data passed to view via Model or ModelAndView objects.

Spring Boot @RestController:

- Combination of @Controller + @ResponseBody.
- Returns data (JSON/XML) directly in the HTTP response — no view file needed.
- Used for building RESTful APIs and microservices.
- Client (browser, mobile, another service) receives structured data.

Key Decision:

- Building a web app with server-side HTML? → Use @Controller.
- Building REST API / backend service? → Use @RestController.

